channel2d.py reference

Stage 2 of the 2D multipath simulator: path lists to complex channel responses. Reference for every function and class, with executed examples that use stage 1 and stage 2 together. Companion to paths2d_reference.pdf (stage 1). Channel file format version 1.0. Written 2026-10-20.

Contents

Overview 2

What stage 2 reads from stage 1 2

Channel model and conventions 3

Constants, band presets and materials 4

Bands 5

`BandConfig` . . . 5

Materials and reflection 5

`eta` . . . 5

`fresnel` . . . 6

Gains and synthesis 6

`path_gains` . . . 6

`synthesize` . . . 6

Channel sets 7

`ChannelSet` . . . 7

`build_channels` . . . 7

`save_channels` . . . 8

`load_channels` . . . 8

`make_channels` . . . 8

Time domain and delay spread 9

`cir` . . . 9

`rms_delay_spread` . . . 9

Selection and normalization helpers 9

`row_indices` . . . 9

`stride_for` . . . 10

`ula_groups` . . . 10

`local_normalize` . . . 10

Command line 11

How the builder works 11

Example use cases 11

Limits and pitfalls 17

Overview

What it does. `channel2d.py` reads a PathSet written by `paths2d.py` and turns each receiver's list of geometric paths into a complex channel response H(f) for one frequency band. Reflection coefficients come from the ITU-R P.2040-3 material model through the Fresnel equations, evaluated at the incidence angle stage 1 stored for every bounce. The result is a `ChannelSet` (an array of shape receivers x frequency samples) that is saved to HDF5, so the analysis stage only loads arrays and never repeats ray tracing.

Where it sits.

Step	Code	File
1. Trace paths (frequency independent)	`paths2d.trace_paths`	paths .h5 (`save_pathset`)
2. Build channels (one band, one material set, one polarization)	`channel2d.build_channels` or `make_channels`	channels .h5 (`save_channels`)
3. Analysis (next stage)	`load_channels`, then fits, diversity, ...	figures, tables

What to redo when something changes.

You change	Redo
band, bandwidth, number of frequency samples	stage 2 only (no re-tracing)
wall materials, polarization, spreading convention, precision	stage 2 only
geometry, obstacles, transmitter or receiver positions	stage 1, then stage 2
max_order, max_length	stage 1, then stage 2

Dependencies. `numpy`, `h5py`, `paths2d`; `scipy` only for `local_normalize`. Everything is one module: `import channel2d as c2`.

What stage 2 reads from stage 1

PathSet field	Used for
`length`	delay (length / C0), spreading loss, and the propagation phase
`walls`	which wall each bounce hit; `scene.material` then gives that wall's material
`inc_angle`	incidence angle for the Fresnel coefficient at each bounce
`offsets`	which rows belong to each receiver (`rx_slice`)
`rx_valid`	receivers flagged invalid get NaN rows
`scene.material_names`	names resolved to ITU materials through `material_map`
`meta`	copied into the channel file as `meta['stage1']`
`order, aoa, aod, points`	not used for H; kept for analysis (for example `order` to separate the direct path)

Stage 1 calls used in the examples. `rectangle_room`, `add_box`, `make_rx_grid` (scene and receivers); `trace_paths` returns a `PathSet`; `save_pathset` writes it. Full details are in paths2d_reference.pdf.

Channel model and conventions

```
H(f_n) = sum over paths k of  a_k * exp(-j * 2*pi * f_n * L_k / C0)
a_k    = [ lambda_c / (4*pi*L_k) ] * product over bounces i of Gamma_i(theta_i, fc)
f_n    = fc + (n - N/2) * df,     n = 0 .. N-1,     df = bandwidth / N
```

L_k is the unfolded path length in metres, lambda_c = C0 / fc, Gamma_i the Fresnel reflection coefficient of the wall hit at bounce i, and theta_i its incidence angle from the wall normal (stored by stage 1). The direct path has no Gamma factor.

Item	Convention
Gain convention	Amplitude lambda_c/(4*pi*L) is the free-space (Friis) channel gain for isotropic 0 dBi antennas, so |H|^2 is the power path gain. 1/L spreading (the top-down slice of a 3D point source), not the 1/sqrt(L) of true 2D. `fspl=False` drops the wavelength factor and keeps 1/L.
Phase	exp(-j 2 pi f tau) with the absolute frequency f, so phases are consistent across bands. Double precision by default.
Frequency grid	N samples (N even) spanning the bandwidth; index N//2 is exactly the carrier, so `narrowband()` is H at fc. The grid is a sampling grid, not the OFDM numerology of any standard.
Alias window	The impulse response repeats every 1/df. Choose N so that C0/df exceeds the longest path (the code does not check this; max_length in stage 1 helps).
Materials and spreading	Evaluated at the carrier only. Fine while the bandwidth is small compared with fc (400 MHz at 13 GHz is about 3 percent).
Antennas	Isotropic, no patterns, no mutual coupling, no transmit power or noise.
Time variation	None: the channel is a static snapshot (quasi-static).
Invalid versus shadowed	NaN rows: the receiver cannot exist (inside an obstacle or outside the room). Zero rows: a valid receiver with no path within the traced order. Mask with rx_valid and n_paths > 0 before any statistics.

Fresnel reflection and polarization

```
eps_c = eps_r - j * sigma / (2*pi*f*eps0)          (complex relative permittivity)
TE:  Gamma = (cos(th) - sqrt(eps_c - sin(th)^2)) / (cos(th) + sqrt(eps_c - sin(th)^2))
TM:  Gamma = (eps_c*cos(th) - sqrt(eps_c - sin(th)^2)) / (eps_c*cos(th) + sqrt(eps_c - sin(th)^2))
```

TE means the E field is perpendicular to the plane of incidence. In a top-down floor plan that is the vertical direction, so TE corresponds to vertically polarized antennas, and the scalar sum over paths is exact (a vertical dipole sees the same E direction on every path). This is the default. **TM** means E lies in the floor-plan plane. Its coefficient is defined in a basis tied to each ray, but the code does not project each path's E vector onto a fixed receive-antenna orientation, so TM is a sensitivity check, not a faithful model of horizontally polarized antennas. Limits to remember: a perfect conductor gives -1 (TE) and +1 (TM); both magnitudes tend to 1 at grazing incidence, which is why long hallways keep many strong reflections; TM has a Brewster dip.

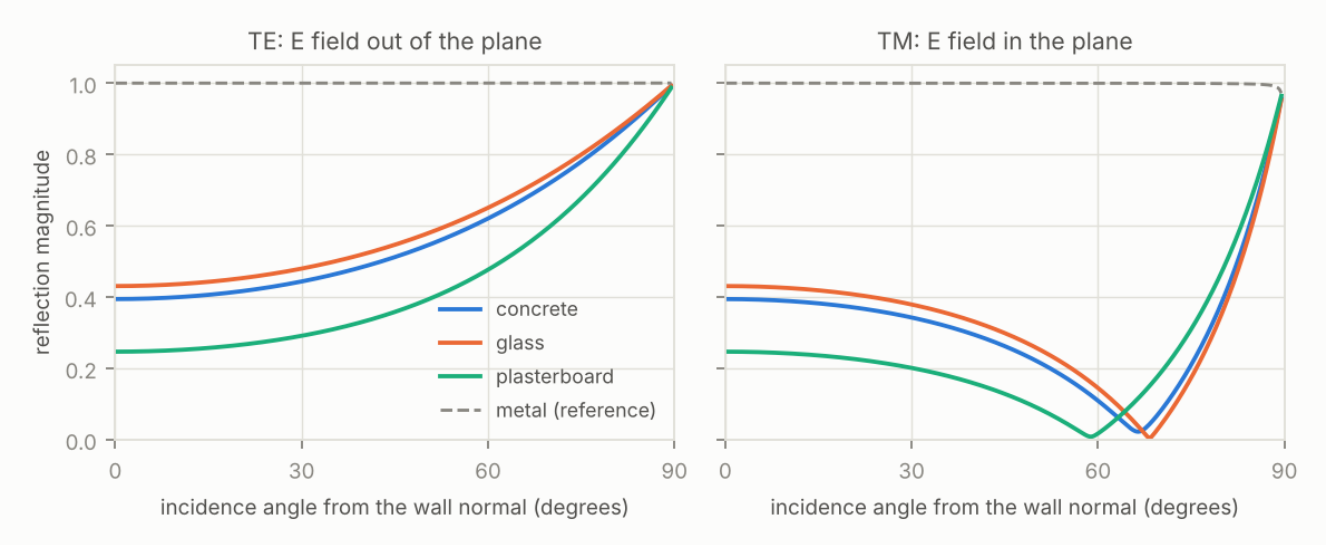


Figure. Reflection magnitude versus incidence angle at 5.5 GHz. Values for specific angles are printed in Example 2.

Constants, band presets and materials

Name	Value	Meaning
`C0`	299792458.0	Speed of light in m/s (same as paths2d.C0).
`EPS0`	8.8541878128e-12	Vacuum permittivity in F/m.
`FORMAT_VERSION`	"1.0"	Channel-file layout version. `load_channels` rejects a different major number.
`DEFAULT_MATERIAL_MAP`	{"wall": "concrete"}	Scenes built with `rectangle_room` name their material "wall"; it maps to concrete unless you override it.

BANDS presets

Preset	fc (GHz)	B (MHz)	N	df (kHz)	lambda (mm)	1/B (ns)	Alias window 1/df
`nr_n78`	3.5	100	256	390.6	85.7	10.0	2.56 us (767 m)
`wifi_5ghz`	5.5	80	256	312.5	54.5	12.5	3.20 us (959 m)
`upper_midband_13ghz`	13	400	512	781.2	23.1	2.5	1.28 us (384 m)
`nr_n257_28ghz`	28	400	512	781.2	10.7	2.5	1.28 us (384 m)

The presets are illustrative; edit them or add your own `BandConfig`. 1/B is the time resolution of the impulse response (about 1.5 times worse with the Hann window).

ITU-R P.2040-3 materials (the ITU dictionary)

Relative permittivity eps_r = a * f^b and conductivity sigma = c * f^d (f in GHz, sigma in S/m). The tuple stored is (a, b, c, d, f_min, f_max); the range is indicative, so `eta` warns outside it. Coefficients are from Table 3 of ITU-R P.2040-3 as reproduced in the MathWorks documentation for `buildingMaterialPermittivity`.

Material	a	b	c	d	Range (GHz)	eps_r at 5.5 GHz	sigma at 5.5 GHz (S/m)
`vacuum`	1	0	0	0	0.001 to 100	1	0

Material	a	b	c	d	Range (GHz)	eps_r at 5.5 GHz	sigma at 5.5 GHz (S/m)
concrete	5.24	0	0.0462	0.7822	1 to 100	5.24	0.175
brick	3.91	0	0.0238	0.16	1 to 10	3.91	0.0313
plasterboard	2.73	0	0.0085	0.9395	1 to 100	2.73	0.0422
wood	1.99	0	0.0047	1.0718	0.001 to 100	1.99	0.0292
glass	6.31	0	0.0036	1.3394	0.1 to 100	6.31	0.0353
ceiling_board	1.48	0	0.0011	1.075	1 to 100	1.48	0.00688
chipboard	2.58	0	0.0217	0.78	1 to 100	2.58	0.082
plywood	2.71	0	0.33	0	1 to 40	2.71	0.33
marble	7.074	0	0.0055	0.9262	1 to 60	7.07	0.0267
floorboard	3.66	0	0.0044	1.3515	50 to 100	outside range	outside range
metal	1	0	1e+07	0	1 to 100	1	1e+07

Bands

`BandConfig`

```
@dataclass(frozen=True)
class BandConfig:
    name, fc, bandwidth, n_sc
```

The frequency plan of one channel simulation. Immutable, so it can be shared and compared; it is stored in the channel file and rebuilt on load.

Parameter	Type	Meaning
name	str	Label used in file names and plots.
fc	float	Carrier frequency in Hz.
bandwidth	float	Simulated bandwidth in Hz.
n_sc	int	Number of frequency samples; must be even (ValueError otherwise) so that the carrier sits exactly on index n_sc // 2.

Notes. Properties and methods: `delta_f` (bandwidth / n_sc, Hz), `wavelength` (C0 / fc, m), `freqs()` (absolute frequency grid, shape (n_sc,), fc + (n - n_sc/2) * delta_f).

Materials and reflection

`eta`

```
eta(material, f_hz) -> complex
```

Complex relative permittivity eps_r - j*sigma/(2*pi*f*eps0) of a material at one frequency.

Parameter	Type	Meaning
material	str or (eps_r, sigma)	An ITU key from `ITU`, or explicit constants (relative permittivity, conductivity in S/m), which skip the table.
f_hz	float	Frequency in Hz.

Returns. A complex number (real part eps_r, negative imaginary part for lossy materials).

Notes. An unknown string raises KeyError listing the known keys. A frequency outside the material's ITU range issues a warning (not an error). The tuple form never warns.

`fresnel`

```
fresnel(theta, eta_c, pol="TE") -> complex ndarray
```

Fresnel reflection coefficient of a flat air-to-material interface (formulas in Conventions).

Parameter	Type	Meaning
`theta`	array-like	Incidence angle from the wall normal, radians (0 = normal incidence).
`eta_c`	complex	Complex relative permittivity of the wall (from `eta`).
`pol`	"TE" or "TM"	E perpendicular to / in the plane of incidence. Anything else raises ValueError.

Returns. Complex coefficient(s) with the shape of theta. Magnitude never exceeds 1 for passive materials.

Notes. Checked limits: normal incidence (1 - n)/(1 + n) for TE and the opposite sign for TM; perfect conductor -1 (TE) / +1 (TM); TM vanishes at the Brewster angle of a lossless dielectric; both tend to -1 at grazing incidence.

Gains and synthesis

`path_gains`

```
path_gains(ps, band, material_map=None, pol="TE", fspl=True) -> (P,) complex
```

Complex amplitude a_k of every path in a PathSet, without the propagation phase. For each bounce column, paths are grouped by the material of the wall hit and multiplied by that material's Fresnel coefficient at the stored incidence angle; the result is divided by the path length and (if `fspl`) scaled by lambda_c/(4*pi).

Parameter	Type	Meaning
`ps`	PathSet	Stage 1 output.
`band`	BandConfig	Reflection and spreading are evaluated at band.fc.
`material_map`	dict or None	Scene material name -> ITU key or (eps_r, sigma). Merged over {"wall": "concrete"}. A scene material whose name is itself an ITU key needs no entry.
`pol`	"TE" or "TM"	Polarization (see Conventions).
`fspl`	bool	True: amplitude lambda_c/(4*pi*L) (true link gain). False: 1/L.

Returns. Array aligned with the per-path arrays of ps (same row order).

Notes. Use this directly when you only need gains or delay spread, or to build H yourself (Example 6).

`synthesize`

```
synthesize(lengths, gains, freqs) -> (N,) complex
```

The core formula: H(f) = sum_k gains_k * exp(-j*2*pi*f*lengths_k/C0) for one receiver.

Parameter	Type	Meaning
`lengths`	(n,) array-like	Path lengths in metres.
`gains`	(n,) complex	Path amplitudes (from `path_gains`).
`freqs`	(N,) array-like	Absolute frequencies in Hz.

Returns. (N,) complex array; zeros if n is 0.

Notes. Builds an n x N complex matrix internally, so memory scales with paths times frequency samples. A single frequency (the carrier) is cheap: pass `[band.fc]`.

Channel sets

`ChannelSet`

```
@dataclass
class ChannelSet:
    H, freqs, rx, rx_valid, n_paths, tx, band, meta
```

Complex channels of one transmitter and many receivers for one band.

Parameter	Type	Meaning
`H`	(R, N) complex	Channel per receiver and frequency. NaN rows: invalid receiver. Zero rows: valid receiver with no path.
`freqs`	(N,) float	Absolute frequencies in Hz; index N//2 is the carrier.
`rx`	(R, 2) float	Receiver positions in metres.
`rx_valid`	(R,) bool	False for receivers inside obstacles or outside the room.
`n_paths`	(R,) int	Number of paths used for each receiver.
`tx`	(2,) float	Transmitter position in metres.
`band`	BandConfig	The frequency plan.
`meta`	dict	pol, fspl, material_map (scene name -> ITU key or [eps_r, sigma]), stage1 (the stage 1 settings), gain_convention; after loading also source.

Notes. Methods: `narrowband()` returns the (R,) complex channel at the carrier; `envelope()` returns its magnitude; `wideband_gain_db()` returns 10*log10 of the frequency-averaged |H|^2 per receiver (minus infinity for shadowed receivers, NaN for invalid ones). Always mask before computing statistics: `ok = ch.rx_valid & (ch.n_paths > 0)`.

`build_channels`

```
build_channels(ps, band, material_map=None, pol="TE", fspl=True,
               dtype=np.complex128, progress=False) -> ChannelSet
```

Turns a PathSet into a ChannelSet for one band: computes `path_gains` once, then calls `synthesize` for every valid receiver.

Parameter	Type	Meaning
`ps`	PathSet	Stage 1 output.
`band`	BandConfig	Frequency plan.
`material_map, pol, fspl`	as in path_gains	Material resolution, polarization, spreading convention.
`dtype`	numpy complex dtype	complex128 (default) or complex64 to halve memory. Delays are applied in double precision before casting.
`progress`	bool	Print coarse progress every 10 percent of receivers.

Returns. A ChannelSet. The resolved material map and the stage 1 settings are recorded in meta.

Notes. Memory is R x N x 16 bytes for complex128 (about 210 MB for 26,000 receivers and 512 samples). For very large sets, build in chunks of receivers or use complex64. Measured on the development container: about 0.7 s for 2,173 receivers at 256 samples including file input and output.

save_channels

```
save_channels(filename, ch, source=None) -> None
```

Writes a ChannelSet to HDF5 (gzip level 4 with shuffle). An existing file is overwritten.

Parameter	Type	Meaning
filename	str	Output path.
ch	ChannelSet	Channels to save.
source	str or None	Name of the stage 1 file the channels came from (recorded as an attribute).

load_channels

```
load_channels(filename) -> ChannelSet
```

Reads a file written by save_channels and rebuilds the ChannelSet, including the BandConfig and meta (with meta["source"]).

Parameter	Type	Meaning
filename	str	HDF5 file to read.

Returns. A ChannelSet identical to the one saved.

Notes. Raises ValueError if the file's format major version differs from this module's.

make_channels

```
make_channels(paths_file, band, out_file, **kw) -> ChannelSet
```

Pipeline convenience: load a stage 1 file, build channels, save them, and return them.

Parameter	Type	Meaning
paths_file	str	Stage 1 HDF5 file.
band	BandConfig	Frequency plan.
out_file	str	Channel HDF5 file to write.
**kw	keyword arguments	Passed to build_channels (material_map, pol, fspl, dtype, progress).

Returns. The ChannelSet that was saved.

Channel file layout

Location	Contents
/ (attributes)	format_version, created_utc (ISO string), band_json (BandConfig), meta_json, source
/H	(R, N) complex, gzip. Stored as an HDF5 compound type with fields r and i; h5py maps it to numpy complex automatically, other readers typically see the two fields.
/freqs	(N,) absolute frequencies in Hz
/rx	xy (R, 2), valid (R,) bool, n_paths (R,) int
/tx	xy (2,)

Time domain and delay spread

`cir`

```
cir(H, band, nfft=None, window="hann") -> (t, h)
```

Channel impulse response by inverse FFT over the frequency samples.

Parameter	Type	Meaning
`H`	(N,) or (R, N) complex	Channel(s) on the band's frequency grid.
`band`	BandConfig	Supplies N and the frequency spacing.
`nfft`	int or None	Output length (at least N). Zero-padding in frequency interpolates in time; it does not add resolution. Default N.
`window`	"hann" or None	"hann" lowers sidelobes; its peak amplitude is compensated. None gives a rectangular window.

Returns. t: (nfft,) delays in seconds, 0 to (nfft-1)/(nfft*df); h: (R, nfft) complex.

Notes. A single path of amplitude a peaks at |h| = a (exactly when its delay falls on the time grid). h is complex baseband, so it includes the carrier phase. Delays beyond 1/df alias back. Resolution is about 1/B (about 1.5/B with the window); closer paths merge (Example 5).

`rms_delay_spread`

```
rms_delay_spread(ps, gains) -> (R,) float
```

RMS delay spread per receiver in seconds, from the discrete paths, power-weighted by |gain|^2.

Parameter	Type	Meaning
`ps`	PathSet	Stage 1 output (only lengths and receiver slices are used).
`gains`	(P,) complex	From `path_gains`.

Returns. (R,) array; NaN where a receiver has no paths.

Notes. Exact for the ray model: unlike a delay spread measured from the impulse response it does not depend on bandwidth. A common derived figure is coherence bandwidth of about 1/(5 * rms delay spread).

Selection and normalization helpers

Small utilities that prepare data for the analysis stage.

`row_indices`

```
row_indices(rx, y) -> int array
```

Indices of the receivers on the grid row nearest height y, ordered by increasing x.

Parameter	Type	Meaning
`rx`	(R, 2) array	Receiver positions (for example ch.rx).
`y`	float	Target height in metres; the nearest existing row is used.

Returns. Index array; use it as ch.H[idx], ch.narrowband()[idx], ...

stride_for

```
stride_for(spacing_m, step_m) -> (stride, actual_spacing_m, relative_error)
```

Grid stride that best realizes a physical spacing on a grid of step step_m.

Parameter	Type	Meaning
spacing_m	float	Wanted spacing in metres (for example lambda/2).
step_m	float	Grid step in metres.

Returns. stride (at least 1), the spacing it gives, and the relative error.

Notes. A coarse grid cannot realize lambda/4 exactly (Example 10); check the error and refine the grid if it matters.

ula_groups

```
ula_groups(n_points, stride, n_elem, hop=1) -> (G, n_elem) int array
```

Every uniform-linear-array placement along a row of n_points receivers. Element j of placement g is row position g*hop + j*stride.

Parameter	Type	Meaning
n_points	int	Number of receivers on the row.
stride	int	Samples between elements (from stride_for).
n_elem	int	Elements per array.
hop	int	Step between successive placements; larger values thin the placements (fewer, less overlapping samples).

Returns. Row positions; map them to receiver indices with row_indices(...)[groups].

local_normalize

```
local_normalize(env, win) -> ndarray
```

Divides an envelope by its local rms over a window of win samples, removing slow (shadowing and path-loss) variation so small-scale fading remains.

Parameter	Type	Meaning
env	1D or 2D array	Envelope; reshape a grid to (ny, nx) first.
win	int	Window size in samples, applied along each axis. Choose it to span roughly 10 to 20 wavelengths.

Returns. Array of the same shape with local mean square about 1; zero where the local rms is zero.

Notes. Uses a uniform filter with reflected edges (needs scipy). Choose win from the wavelength and your grid step: win = round(10 * wavelength / step).

Command line

`python channel2d.py paths.h5 band_name out.h5 [TE|TM]` loads a stage 1 file, builds channels for one preset band and saves them. Executed below on the stage 1 demo file.

Commands:

```
python paths2d.py
python channel2d.py demo_paths.h5 wifi_5ghz demo_ch.h5 TE
```

Last line of output:

```
wifi_5ghz: 2173 receivers, 2173 with signal, narrowband gain -90.9 to -58.0 dB
```

How the builder works

1. Gains. `path_gains` starts from ones and, for each bounce column, multiplies in the Fresnel coefficient of the wall's material at the stored incidence angle, grouped by material so each group is one vectorized call. It then divides by the path length and applies lambda_c/(4*pi).

2. Synthesis. `build_channels` slices the flat path arrays per receiver (`rx_slice`) and evaluates the sum over paths at all N frequencies with one matrix product. Invalid receivers are skipped and stay NaN.

3. Packaging. The ChannelSet carries positions, validity, path counts and the settings, so a channel file is self-describing and needs nothing from the stage 1 file to be analysed.

Not modelled. Antenna patterns and polarization mixing, frequency-dependent materials across the band, noise and transmit power, motion (Doppler), diffraction, diffuse scattering and transmission through obstacles (stage 1 has no such path types).

Example use cases

Every snippet below was executed against this version of `channel2d.py` and `paths2d.py` and the output shown is the real output. Save each as a script next to the two modules.

Example 1: the smallest stage 1 + stage 2 run

One room, one receiver. Stage 1 finds the paths, stage 2 turns them into a channel. The narrowband gain with all paths is compared with the direct path alone (Friis); the difference is the multipath contribution at this one point.

```
import numpy as np
import paths2d as p2
import channel2d as c2

room = p2.rectangle_room(10.0, 8.0)
ps = p2.trace_paths(room, (2.0, 3.0), [[8.0, 5.0]], max_order=3)      # stage 1
band = c2.BANDS["wifi_5ghz"]
ch = c2.build_channels(ps, band)                                       # stage 2

d = np.hypot(6.0, 2.0)                                                 # direct distance (m)
friis = 20 * np.log10(band.wavelength / (4 * np.pi * d))
print("paths:", ch.n_paths[0], "  H shape:", ch.H.shape)
print("narrowband gain, all paths (dB):", round(20 * np.log10(ch.envelope()[0]), 2))
print("direct path alone, Friis (dB):  ", round(friis, 2))
```

Output:

```
paths: 25   H shape: (1, 256)
narrowband gain, all paths (dB): -59.98
direct path alone, Friis (dB):   -63.28
```

Example 2: materials and Fresnel reflection

`eta` gives the complex permittivity of an ITU-R P.2040 material at a frequency; `fresnel` turns it into a reflection coefficient. The table printed here is the numeric version of the figure in the Conventions section. Asking for a material outside its ITU frequency range warns instead of failing.

```
import warnings
import numpy as np
import channel2d as c2

degs = [0, 30, 60, 80, 89]
theta = np.radians(degs)                        # incidence angle from the wall normal
print("incidence angle (deg):", degs)
for m in ("concrete", "glass", "plasterboard", "metal"):
    e = c2.eta(m, 5.5e9)
    te = np.abs(c2.fresnel(theta, e, "TE"))
    tm = np.abs(c2.fresnel(theta, e, "TM"))
    print(f"{m:<13} eps_c = {e.real:6.2f} {e.imag:+.4f}j")
    print("   |TE|", np.round(te, 3), "  |TM|", np.round(tm, 3))

with warnings.catch_warnings(record=True) as caught:     # outside the ITU range: warns only
    warnings.simplefilter("always")
    c2.eta("brick", 13e9)
print("warning:", caught[0].message)
```

Output:

```
incidence angle (deg): [0, 30, 60, 80, 89]
concrete      eps_c =   5.24 -0.5729j
   |TE| [0.394 0.443 0.62  0.846 0.983]   |TM| [0.394 0.342 0.109 0.389 0.915]
glass         eps_c =   6.31 -0.1154j
   |TE| [0.431 0.48  0.65  0.86  0.985]   |TM| [0.431 0.379 0.145 0.357 0.909]
plasterboard  eps_c =   2.73 -0.1378j
   |TE| [0.247 0.291 0.476 0.769 0.974]   |TM| [0.247 0.201 0.017 0.474 0.93 ]
metal         eps_c =   1.00 -32682006.5173j
   |TE| [1. 1. 1. 1. 1.]   |TM| [1.    1.    1.    0.999 0.986]
warning: brick: 13 GHz is outside the ITU-R P.2040 range [1, 10] GHz
```

Example 3: one path, free-space loss and delay

`synthesize` is the core formula. A single path must give exactly the Friis gain, flat across the band, and its delay must be recoverable from the phase slope versus frequency.

```
import numpy as np
import channel2d as c2

band = c2.BANDS["wifi_5ghz"]
d = 12.0                                                # one path, 12 m long
H = c2.synthesize([d], [band.wavelength / (4 * np.pi * d)], band.freqs())

friis = 20 * np.log10(band.wavelength / (4 * np.pi * d))
gain = 20 * np.log10(np.abs(H[[0, 128, 255]]))          # three samples across the band
print("gain at band start/middle/end (dB):", np.round(gain, 3).tolist())
print("Friis (dB):                        ", round(friis, 3))
step = np.angle(H[1:] / H[:-1])                         # phase step per frequency sample
print("delay from phase slope (ns):", round(-step.mean() / (2 * np.pi * band.delta_f) * 1e9, 3))
print("true delay (ns):            ", round(d / c2.C0 * 1e9, 3))
```

Output:

```
gain at band start/middle/end (dB): [-68.839, -68.839, -68.839]
Friis (dB):                          -68.839
delay from phase slope (ns): 40.028
true delay (ns):             40.028
```

Example 4: two paths and comb-filter nulls

Two equal paths 3 m apart in length interfere destructively wherever the frequency times the delay difference is a half-integer. This is the simplest check that phases are applied at the true frequency.

```
import numpy as np
import channel2d as c2

f = np.arange(5.40e9, 5.60e9 + 1, 1e6)                    # 1 MHz grid across 200 MHz
H = c2.synthesize([30.0, 33.0], [1.0, 1.0], f)            # second path is 3 m longer
mag = np.abs(H)
nulls = f[1:-1][(mag[1:-1] < mag[:-2]) & (mag[1:-1] < mag[2:])]
dtau = 3.0 / c2.C0
print("null frequencies (GHz):", np.round(nulls / 1e9, 3).tolist())
pred = [round((n + 0.5) / dtau / 1e9, 4) for n in (54, 55)]
print("predicted (n + 1/2) / dtau for n = 54, 55:", pred)
```

Output:

```
null frequencies (GHz): [5.446, 5.546]
predicted (n + 1/2) / dtau for n = 54, 55: [5.4462, 5.5462]
```

Example 5: impulse response and bandwidth

Two paths 10 ns apart. The impulse response can only separate them if the band is wide enough (rule of thumb: resolution about 1/B, somewhat worse with the Hann window). This is why the same geometry looks like one tap at 80 MHz and two at 400 MHz.

```
import numpy as np
import channel2d as c2

for name in ("wifi_5ghz", "upper_midband_13ghz"):
    band = c2.BANDS[name]
    H = c2.synthesize([30.0, 33.0], [1.0, 0.8], band.freqs())     # 3 m = 10 ns apart
    t, h = c2.cir(H, band, nfft=8 * band.n_sc)                     # Hann window, 8x interpolation
    mag = np.abs(h[0])
    pk = np.flatnonzero((mag[1:-1] > mag[:-2]) & (mag[1:-1] > mag[2:]) & (mag[1:-1] > 0.3)) + 1
    print(f"{name}: B = {band.bandwidth / 1e6:.0f} MHz, 1/B = {1e9 / band.bandwidth:.1f} ns, "
          f"{len(pk)} peak(s) at {np.round(t[pk] * 1e9, 1).tolist()} ns")
```

Output:

```
wifi_5ghz: B = 80 MHz, 1/B = 12.5 ns, 1 peak(s) at [104.7] ns
upper_midband_13ghz: B = 400 MHz, 1/B = 2.5 ns, 2 peak(s) at [100.0, 110.0] ns
```

Example 6: a different material on every wall

Materials live in the scene (an index per wall) and are resolved in stage 2. Names that match an ITU key need no mapping. A `material_map` entry can rename a material to another ITU key or give explicit constants `(eps_r, sigma)`. The printed magnitudes are the reflection coefficients (the gain divided by the spreading loss).

```
import numpy as np
import paths2d as p2
import channel2d as c2

room = p2.rectangle_room(10.0, 8.0, material=0, material_names=("concrete", "glass", "metal"))
room.material[[0, 2]] = 1                   # bottom and top walls: glass
room.material[3] = 2                        # left wall: metal
ps = p2.trace_paths(room, (2.0, 3.0), [[8.0, 5.0]], max_order=1)
band = c2.BANDS["wifi_5ghz"]
g = c2.path_gains(ps, band)
lossless = band.wavelength / (4 * np.pi * ps.length)         # spreading loss only
for i in range(len(g)):
    w = int(ps.walls[i, 0])
    name = "direct" if w < 0 else room.material_names[room.material[w]]
    ang = "   -  " if w < 0 else f"{np.degrees(ps.inc_angle[i, 0]):5.1f} "
    print(f"wall {w:2d}  {name:<9} incidence {ang}deg  |Gamma| = {abs(g[i]) / lossless[i]:.3f}")

mm = {"concrete": (4.0, 0.0), "glass": "plasterboard"}        # rename / explicit constants
g2 = c2.path_gains(ps, band, material_map=mm)
change_db = 20 * np.log10(np.abs(g2[1:] / g[1:]))
print("gain change of the 4 reflections (dB):", np.round(change_db, 2).tolist())
```

Output:

```
wall -1  direct    incidence    -  deg  |Gamma| = 1.000
wall  0  glass     incidence  36.9 deg  |Gamma| = 0.506
wall  2  glass     incidence  36.9 deg  |Gamma| = 0.506
wall  1  concrete  incidence  11.3 deg  |Gamma| = 0.401
wall  3  metal     incidence  11.3 deg  |Gamma| = 1.000
gain change of the 4 reflections (dB): [-4.07, -4.07, -1.43, 0.0]
```

Example 7: files in and out, and the one-line pipeline

`make_channels` is load, build and save in one call. The channel file records its source, polarization, material map and the stage 1 settings, so it is self-describing.

```
import numpy as np
import paths2d as p2
import channel2d as c2

room = p2.rectangle_room(10.0, 8.0)
ps = p2.trace_paths(room, (2.0, 3.0), [[8.0, 5.0], [3.0, 6.0]], max_order=3)
p2.save_pathset("tiny_paths.h5", ps)                                    # stage 1 file

band = c2.BANDS["nr_n78"]
ch = c2.make_channels("tiny_paths.h5", band, "tiny_n78.h5", pol="TM")     # load + build + save
back = c2.load_channels("tiny_n78.h5")
print("identical after reload:", np.array_equal(ch.H, back.H))
print("band:", back.band)
print("source:", back.meta["source"], "  pol:", back.meta["pol"])
print("material_map:", back.meta["material_map"])
print("stage 1 settings kept:", {k: back.meta["stage1"][k] for k in ("max_order", "max_length")})
```

Output:

```
identical after reload: True
band: BandConfig(name='nr_n78', fc=3500000000.0, bandwidth=100000000.0, n_sc=256)
source: tiny_paths.h5   pol: TM
material_map: {'wall': 'concrete'}
stage 1 settings kept: {'max_order': 3, 'max_length': None}
```

Example 8: invalid receivers versus shadowed receivers

Rows of `H` mean different things: NaN marks a receiver that cannot exist (inside an obstacle), zero marks a real receiver with no path at all. Build a usable mask before any statistics.

```
import numpy as np
import paths2d as p2
import channel2d as c2

scene = p2.add_box(p2.rectangle_room(10.0, 8.0), 4.0, 3.0, 6.0, 5.0)
rx = [[1.0, 1.0], [5.0, 4.0], [9.0, 4.0]]          # free, inside the box, behind the box
ps = p2.trace_paths(scene, (1.0, 4.0), rx, max_order=0)       # direct paths only
ch = c2.build_channels(ps, c2.BANDS["wifi_5ghz"])

print("rx_valid:", ch.rx_valid.tolist(), "  n_paths:", ch.n_paths.tolist())
print("row 1 all NaN:", bool(np.isnan(ch.H[1]).all()))
print("row 2 all zero:", bool((ch.H[2] == 0).all()))
ok = ch.rx_valid & (ch.n_paths > 0)
print("usable receivers:", np.flatnonzero(ok).tolist())
```

Output:

```
rx_valid: [True, False, True]   n_paths: [1, 0, 0]
row 1 all NaN: True
row 2 all zero: True
usable receivers: [0]
```

Example 9: delay spread and impulse response, line of sight versus shadow

One receiver in line of sight and one behind the box, at two bands. Behind the box the direct path is blocked, so the first arrival is a reflection (slightly later than the blocked direct path) and the rms delay spread about doubles. `rms_delay_spread` works from the discrete paths, so it does not depend on bandwidth (the small difference between bands comes from the materials at the two carriers). The strongest impulse-response tap is blurred by the bandwidth, so it only approximates the exact first arrival.

```
import numpy as np
import paths2d as p2
import channel2d as c2

scene = p2.add_box(p2.rectangle_room(20.0, 3.0), 8.0, 0.8, 9.0, 2.2)
rx = np.array([[5.0, 1.5], [12.0, 1.5]])
ps = p2.trace_paths(scene, (2.0, 1.5), rx, max_order=4)
direct = np.linalg.norm(rx - [2.0, 1.5], axis=1) / c2.C0 * 1e9          # blocked for rx 1
first = [ps.length[ps.rx_slice(i)][0] / c2.C0 * 1e9 for i in range(2)]   # first path found
print("receivers: line of sight, behind the box")
print("direct-path delay (ns):", np.round(direct, 1))
print("first arrival (ns):    ", np.round(first, 1))
for name in ("wifi_5ghz", "upper_midband_13ghz"):
    band = c2.BANDS[name]
    ch = c2.build_channels(ps, band)
    ds = c2.rms_delay_spread(ps, c2.path_gains(ps, band)) * 1e9
    t, h = c2.cir(ch.H, band, nfft=8 * band.n_sc)
    peak = t[np.argmax(np.abs(h), axis=1)] * 1e9
    print(f"{name}: rms delay spread (ns) {np.round(ds, 1)}")
    print(f"{'':<20}strongest CIR tap (ns) {np.round(peak, 1)}")
```

Output:

```
receivers: line of sight, behind the box
direct-path delay (ns): [10.  33.4]
first arrival (ns):     [10.  34.8]
wifi_5ghz: rms delay spread (ns) [ 5.2 10. ]
                    strongest CIR tap (ns) [ 9.4 35.9]
upper_midband_13ghz: rms delay spread (ns) [5.2 9.9]
                    strongest CIR tap (ns) [ 9.7 34.7]
```

Example 10: row selection, array placements and local normalization

From one fine row of receivers: pick the row, convert a half-wavelength spacing into a grid stride (with its rounding error), list every 4-element array placement, extract the (placements x elements) channel matrix, and normalize the envelope by its local rms.

```
import numpy as np
import paths2d as p2
import channel2d as c2

scene = p2.add_box(p2.rectangle_room(20.0, 3.0), 8.0, 0.8, 9.0, 2.2)
band = c2.BANDS["wifi_5ghz"]
pts, xs, _ = p2.make_rx_grid((4.0, 4.5), (1.5, 1.5), 0.002)          # 251 receivers, 2 mm apart
ps = p2.trace_paths(scene, (2.0, 1.5), pts, max_order=3)
ch = c2.build_channels(ps, band)

row = c2.row_indices(ch.rx, 1.5)                                      # row y = 1.5, ordered by x
stride, actual, err = c2.stride_for(band.wavelength / 2, 0.002)       # half-wavelength spacing
print(f"stride {stride} samples = {actual * 1e3:.1f} mm (error {err * 100:+.1f} %)")
groups = c2.ula_groups(len(row), stride, n_elem=4, hop=20)
h_arr = ch.narrowband()[row][groups]                                  # (placements, elements)
print("array placements:", groups.shape[0], "  channel matrix:", h_arr.shape)
print("first placement uses row samples:", groups[0].tolist())

env = c2.local_normalize(ch.envelope()[row], win=125)                 # about 4.6 wavelengths
print("mean square of normalized envelope:", round(float(np.mean(env ** 2)), 3))
```

Output:

```
stride 14 samples = 28.0 mm (error +2.7 %)
array placements: 11   channel matrix: (11, 4)
first placement uses row samples: [0, 14, 28, 42]
mean square of normalized envelope: 1.01
```

Limits and pitfalls

Topic	What to know
Polarization	TE is exact for vertical antennas. TM is a scalar approximation without antenna orientation projection; use it only as a sensitivity check.
Default material	A scene from rectangle_room has material "wall", which maps to concrete. Check material_map before trusting absolute levels.
ITU frequency range	eta warns outside a material's range (brick is quoted only up to 10 GHz, floorboard only 50 to 100 GHz). Treat warned results as extrapolation.
Alias window	Not checked. If C0/df is shorter than the longest path, late paths wrap into early delays in the impulse response (H itself is unaffected).
Absolute level	Isotropic antennas, no transmit power or noise. H is a path gain; add power and noise in analysis.
Grid spacing	To resolve the fading pattern use receiver spacing of lambda/4 or finer (2.7 mm at 28 GHz). A coarse grid aliases the pattern.
Invalid versus zero	NaN rows are not receivers; zero rows are receivers with no path. Mask both (rx_valid and n_paths > 0) before statistics.
Hard shadows	Stage 1 has no diffraction, so shadow regions have abrupt on/off paths and the channel jumps there. This carries into every H computed from them.
Memory	R x N x 16 bytes (complex128). Use complex64 or chunking for very large sets. complex64 rounding (about 1e-7 relative) is harmless for envelopes.
Reproducibility	Channel files record band, polarization, material map and stage 1 settings; keep the paths file name in the source attribute and name files after the varied parameter.
scipy	Only local_normalize needs it (imported inside the function).